# A different approach to RMP

How to make the world control your robot for you



ALP ORAN

OCT 27, 2026

## Foreword

Differential geometry at a glance looks intimidating, and in all fairness it is intimidating. I personally don’t have a background in mathematics, due to this I am unfamiliar with a lot of the language and concepts used to describe differential geometry. Things like tensor calculus and Christoffel symbols are the preferred language for this but I don’t speak this fluently enough to fully digest the things it is expressing. However what I am more familiar with are things like classical mechanics and robotics. This blog discusses a way I found that made sense to me in terms of things I understood. I don’t claim to have invented a totally new way to do stuff but I believe for my purpose there are some advantages (especially computational) to do it this way. However that is not to say I just used stuff I knew prior. Through this process I taught myself a lot of different concepts (not usually applied to robotics in the way I do, I borrowed a lot of concepts from gauge theory which is from general relativity). In the blog ,assuming people are starting in a similar starting point that I started at, I discuss how these “newly acquired” concepts work, as well as build on them for the context of robotics

## A curved (Riemannian) world’s notation

Before continuing we need to establish some basic concepts. The most principle of which is what a “metric” is. A metric is basically a measure of how something warps, stretches or rotates based on our input. The metric in its most simple form can be though of as a dot product of tangent vectors:

$$g_{ij} = e_i \cdot e_j$$

Consider that the vectors have unit length, this means the individual values of g are a measure of the angles of these tangent vectors on a given tangent space. The dot product is a symmetric operation meaning:

$$g_{ij} = g_{ji}$$

Another feature of the dot product is that a real tangent vector’s dot product with itself is always positive, thus:

$$g_{ii} = e_i \cdot e_i = 1 > 0$$

If we define a function g(x,y), where g(x,y) is the dot product of the tangent vectors x and y, this is expressed as :

$$g(X, Y) = X^T G\, Y$$

The G in the equation is our metric which is the full collection of $g_{ij}$ values. This is necessary since in a curved coordinate frame every axis isn’t perpendiccular.

A cool thing we can consider is g(X,X) which gives the squared value of |X|:

if we consider x to be a differential vector (dx), this gives a squared differential distance:

$$g(dx, dx) = dx \cdot dx = dx^T G dx = ds^2$$

Consider that a Jacobian matrix in robotics maps differential inputs of our input to differential outputs of x:

$$Jdq = dx$$

“dx” in this instance lives in an Euclidian frame where every axis is perpendicular to each other, thus G is just an identity matrix. By substituting this definition for dx we get the following expression:

$$ds^2 = dq^T (J^T J) dq$$

This is equal to the Euclidian distance in world frame in terms of joint frame coordinates. As we can see the term in the middle is our new metric.

# How to use the curved world notation

Because a Riemannian metric is by definition positive definite and symmetric, it can be expressed in terms of “Vielbein”(s), which means “many legged” in German, It is a common way to express the metric in

Gauge Theory which is a way to view general relativity. Gauge Theory basically says that a curved geometry can be though of as just a regular flat Euclidian space, however we change how we measure stuff in that flat space in a way that it would have been equal if we had measured on the curved geometry. In simpler terms we alter our perception of the flat frame to do approximate a curved frame. In Gauge Theory the metric is defined as such:

$$M = L^T L$$

The Vielbein is essentially an embedding, it maps differential changes in our input to a differential change in our output:

$$Ldq = dx$$

As we can see, the Vielbein works exactly like a Jacobian matrix in robotics. Lets take this a step forward. The most common purpose of the Jacobian matrix in robotics is to map joint velocities to end effector velocities. Lets do something similar with the Vielbein.

$$v = L\dot{q}$$

By considering the time derivative of this we get the acceleration

$$a = \dot{L}\dot{q} + L\ddot{q}$$

A trick we can borrow from the principle of virtual work is the fact that there are two ways to express power, one in terms of the original frame and one in the final frame.

$$v \cdot a = P = \dot{q}^T \, \tau$$

$$v = L\dot{q} \ \wedge \ v \cdot a = v^T a \rightarrow \dot{q}^T L^T a = \dot{q}^T \tau$$

This nets us the final expression:

$$\tau = L^T a$$

By multiplying both sides by the transposed Vielbein we get the equation

$$L^T L\ddot{q} + L^T \dot{L}\dot{q} = \tau$$

Which is just the familiar Robot equation in disguise

$$M\ddot{q} + C\dot{q} = \tau$$

More specifically the conventional “Mass” matrix in the robot equation is equal to the following:

$$M = J^T I J$$

I in this context is the inertia matrix which is a tensor in its own right. I is a diagonal matrix which makes expressing it as the multiplication of square roots quite easy. This would make L:

$$M = J^T I^{\frac{1}{2}T} I^{\frac{1}{2}} J \rightarrow L = I^{\frac{1}{2}} J$$

For the context of robotics lets generalize the Vielbein in terms of a Gain co-tensor times the robot Jacobian

$$L = KJ$$

$$G = J^T W J$$

This way of constructing a metric is referred to as a pullback metric

# How to exploit the curved world

For a simple case lets consider one of the rather troublesome aspects of robotics, singularity avoidance. There are several different ways to do this, some create artificial force fields around the singularity points, mostly we just brute force our way by injecting a small diagonal component to the Jacobian when near a singularity so that we don't lock up. What I propose is sort of combination of both, we define the metric itself to be aware of the singularity, and steer itself away automatically. For sake of demonstration lets consider W to be:

$$W = (JJ^T + I\alpha)^{-1}$$

$$K = (JJ^T + I\alpha)^{-0.5}$$

The purpose of this gain co-tensor is to blow up (not to infinity but to a predetermined value controlled by alpha) when we are near singularities. This basically stretches the distances around the singularities in a way that it is straighter to travel around the singularities, think of it like traveling around mountains, mountains represent the singularities, it is easier to travel through the valleys.

When we input our new definition for K we get the following equations

$$L = (JJ^T + I\alpha)^{-0.5}J$$

$$\dot{L} = \dot{K}J + K\dot{J}$$

$$M = J^T(JJ^T + I\alpha)^{-0.5T}(JJ^T + I\alpha)^{-0.5}J = J^TWJ$$

$$C = J^TK^T\dot{K}J + J^TK^TK\dot{J} = J^TC_KJ + J^TW\dot{J}$$

$$\tau = L^Ta = J^TWa_{flat} + J^TC_Kv_{flat}$$

For our final step we need to define how we define acceleration. A typical example can be thought of as follows which is essentially a proportional controller, how close we get to converging is dictated by beta.

$$a_{flat} = (1 - e^{-\beta|error|^2})\frac{error}{|error|}$$

$$a_{curved} = (JJ^T + I\alpha)^{-1}a_{flat}$$

$$\sigma = \frac{(1-e^{-\beta|error|^2})}{|error|}$$

$$\tau = \sigma \times J^TWerror$$

The only concern that is left remaining is the fact that in our Coriolis Matrix definition we have the term Kdot, this is due to the non commutative nature of matrices. Let us define our inverse weight matrix as A.

$$A = (JJ^T + I\alpha)$$

Since we defined K as the square root of a symmetric matrix, K itself is also symmetric, this means that K squared is exactly the inverse of A.

$$K^2 = A^{-1}$$

By leveraging this fact we can take the time derivative of both sides

$$\frac{d}{dt}A^{-1} = \frac{d}{dt}(K^2)$$

$$\dot{A}^{-1} = K\dot{K} + \dot{K}K = Q$$

This equation is referred to as a Sylvester Equation and is notoriously hard to solve for in real time. This is due to the fact that due to the non commutativity of matrices we can't factor anything that would give us a clean value for Kdot. This is where we can leverage 2 numerical attack angles. To get a quick and dirty way, and the semi -analytical way.

The quick and dirty way leverages the fact that at 1kHz our gain co-tensor doesn't change a lot between time steps. We use a Newton Schulz iteration to define what the current K should be based on what A and the previous time step's K was. This method converges quadratically so it is more than capable for what we are doing.

$$K_{k+1} = \frac{1}{2}X_k\left(3I - A_{k+1}K_k^2\right)$$

Because this operation is quick to compute and yet still very accurate we are able to compute finite differences for the time derivative of K

$$\dot{K} = \frac{K_{k+1} - K_k}{\Delta t}$$

The second more analytical way is to compute a half binomial expansion.

$$K = (JJ^T + I\alpha)^{-0.5}$$

We can express this in simpler terms by defining a temporary matrix X and then factoring by alpha.

$$\frac{1}{\alpha}JJ^T = X$$

$$K = (\alpha(X + I))^{-0.5} = \alpha^{-0.5}(X + I)^{-0.5}$$

By using the half binomial expansion for a matrix we can write K as a matrix polynomial

$$(X+I)^{-0.5} = I - \frac{1}{2}X + \frac{3}{8}X^2 - \frac{5}{16}X^3 + \mathcal{O}(X^4)$$

$$\frac{d}{dt}(X+I)^{-0.5} \approx \frac{d}{dt}\left(I - \frac{1}{2}X + \frac{3}{8}X^2 - \frac{5}{16}X^3\right)$$

We are able to compute what Xdot in terms of the J and Jdot which we already know. This gives the expression

$$\dot{X} = \frac{1}{\alpha}\left(\dot{J}J^T + J\dot{J}^T\right)$$

Finally we are able to express Kdot to be approximately equal to the following expression

$$\dot{K} \approx \alpha^{-1/2}\left[-\frac{1}{2}\dot{X} + \frac{3}{8}(\dot{X}X + X\dot{X}) - \frac{5}{16}(\dot{X}X^2 + X\dot{X}X + X^2\dot{X})\right]$$

# Why would we do this

Consider what we do normally do for robotics. The most basic way to tackle numerical IK is using a gradient descent, which is a first order operation. Let us define that we want the end effector travel towards our desired position, we can achieve this by considering our velocity to always point towards our desired point.

$$v = |v|e_v$$

$$e_v = \frac{error}{|error|}$$

By leveraging the fact that the inverse Jacobian when multiplied by a velocity gives us joint velocities we can define how our arm should move so that we minimize error.

$$\dot{q} = J^{-1}v = \frac{|v|}{|error|}J^{-1}error = kJ^{-1}error$$

“|v|” here is basically a slider we are able to play with to determine how fast we move

By considering our joint velocities as a finite difference, we are able to define our future joint angles.

$$\dot{q} \approx \frac{q_{i+1}-q_i}{\Delta t}$$

$$q_{i+1} = q_i + \Delta t\ k_i J_i^{-1} error_i$$

This is a basic gradient descent algorithm to do robotic control. It’s main issue is that when it reaches singularities the Jacobian matrix loses rank (det(J)=0) thus the function’s inverse blows up to infinity which throws everything out of whack. Normally this is approached several different ways. The most simple is injecting a small diagonal component to the Jacobian when it is about to lose rank so that it is always invertible.

$$\alpha(\mu) = \begin{cases} 0 & \mu > \mu_0 \\ \alpha_0 & 0 \leq \mu < \mu_0 \end{cases}$$

$$J' = J + I\alpha(\mu)$$

This is the easiest solution to implement however this is not usually the desired solution because our model starts drifting away from the actual

robot as we get to singularities.
Again another incredibly common solution which I am personally quite fond of is Levenberg-Marquardt algorithm or the Damped Least Squares.

$$J^* = J^T(JJ^T + \lambda I)^{-1}$$

The logic is to introduce a very miniscule diagonal component to the manipulability matrix. This way we aren't directly adding a diagonal component to J, but rather insinuating that if it were manipulable what that inverse would be. This way to take the Jacobian inverse trades is a bit of path accuracy for stability.

However as you might have notices all of these methods are actually first order systems, which even though does give us the correct joint angles for our destination, it doesn't give us usable trajectories to get to said destination. That is why we use 2nd order systems to define paths. This is done via computing Christoffel symbols of the first kind to determine what our Coriolis Matrix is, construct a robot equation with a pullback metric and then just hack away at the problem. There is nothing wrong with this approach but especially computing the Christoffel symbols is a pain.

Thanks for reading! Subscribe for free to receive new posts and support my work.

Subscribed